Avaliação 01: Dossiê de diagnóstico conceitual

Nome completo: Bruno Ribeiro Lima Lins de Araújo
Turma: 8º período
Disciplina: Grandes Modelos de Linguagem: do Transformer aos Agentes de IA (V2) · Aulas 1 a 8

Caso 1: Acerta muito, mas perde as urgências

Problema central. As classes são desbalanceadas, e o objetivo real (priorizar urgências) não é medido pela acurácia.

1. Por que a acurácia engana. A acurácia conta acertos sobre o total. Como quase todas as solicitações são comuns, um classificador que responde "comum" para tudo acerta quase sempre e, mesmo assim, não detecta nenhuma urgência. O número alto reflete a distribuição das classes, não a capacidade de cumprir a tarefa.

2. Erro prioritário e métrica. O erro crítico é o falso negativo: uma urgência rotulada como comum, que deixa um caso grave sem prioridade. A métrica pertinente é a revocação (recall) da classe "urgente": entre as urgências reais, quantas o sistema encontra. Ela expõe exatamente o que a acurácia esconde; a matriz de confusão por classe complementa a leitura.

3. Excesso de falsos positivos. Se muitas solicitações comuns forem marcadas como urgentes, a fila prioritária incha, as urgências reais disputam atenção com alarmes falsos e a equipe passa a desconfiar do sistema. Por isso a decisão não é maximizar só a revocação: acompanha-se também a precisão da classe urgente e escolhe-se o limiar conforme o custo de cada erro e a capacidade da equipe.

Fundamento. Aula 1: trecho sobre métricas de classificação (acurácia, precisão, revocação e matriz de confusão). Externo: Jurafsky e Martin, *Speech and Language Processing*, seção sobre avaliação de classificadores.

Caso 2: Mesmo conteúdo, mais tokens

Problema central. O número de tokens depende do vocabulário do tokenizador, não da quantidade de conteúdo do texto.

1. Origem da diferença. Tokenizadores de subpalavras (como BPE) constroem o vocabulário a partir das sequências mais frequentes no corpus de treino. O que é frequente vira um token inteiro; o que é raro é quebrado em pedaços. Se o corpus foi dominado pelo inglês, palavras inglesas tendem a ocupar um token, enquanto palavras portuguesas com flexões ("cantávamos"), acentos e clíticos são fragmentadas em várias subpalavras. Em tokenizadores por bytes, letras acentuadas ainda ocupam mais bytes.

2. Consequências. O texto em português consome mais da janela de contexto: cabe menos histórico ou documento, e cresce o risco de truncamento. Num serviço cobrado por token, a mesma informação custa mais e gera mais latência, tanto na entrada quanto na saída.

3. Limites do resultado. Não se pode concluir que o português sempre exige mais tokens: houve um par de textos e um único tokenizador. Um tokenizador treinado com mais português pode reduzir ou inverter a diferença, e o domínio, o estilo e a tradução também influem. Uma comparação mais representativa usaria um corpus paralelo amplo, de vários domínios, com vários tokenizadores, medindo a razão de tokens PT/EN por par de sentenças e relatando a distribuição, não um caso isolado.

Fundamento. Aula 2: trecho sobre tokenização por subpalavras; Aula 4: interpretação do laboratório. Externo: Petrov et al. (2023), que comparam o comprimento de traduções equivalentes em 17 tokenizadores.

Caso 3: Uma palavra, dois contextos

Problema central. Um único vetor por palavra não acompanha a mudança de sentido entre frases.

1. Limitação do vetor estático. No Word2Vec, cada palavra do vocabulário corresponde a uma linha de uma tabela de embeddings, consultada pela identidade da palavra, não pela frase. As duas ocorrências de "banco" recebem exatamente o mesmo vetor, que mistura os sentidos de assento e de instituição financeira na proporção em que aparecem no corpus. Olhando só esse vetor, não há como saber qual sentido está em jogo.

2. O que a representação contextual muda. Num modelo contextual, como um encoder Transformer do tipo BERT, o vetor de cada ocorrência é calculado a partir das palavras ao redor. "Sentou" e "praça" levam "banco" a uma região diferente daquela produzida por "abriu" e "conta". A mesma palavra passa a ter uma representação por ocorrência, o que permite distinguir os sentidos.

3. Relação com a aprendizagem pelo contexto. As duas abordagens partem da hipótese distribucional: o significado se aprende pelos contextos de uso. O Word2Vec aplica essa ideia só no treino e resume todos os contextos de "banco" num único vetor. O modelo contextual aplica-a também no uso: o contexto da frase atual entra no cálculo de cada representação.

Fundamento. Aula 3: trecho sobre Word2Vec e hipótese distribucional. Externos: Mikolov et al. (2013); Devlin et al. (2019).

Caso 4: O gráfico convenceu a equipe

Problema central. Uma projeção visual e um corpus pequeno não sustentam uma conclusão sobre compreensão e generalização.

1. Duas limitações.

- **Visualização:** o t-SNE é uma projeção não linear que preserva sobretudo vizinhanças locais. Distâncias entre grupos e tamanhos de agrupamentos não são confiáveis, e o desenho muda com a perplexidade e a semente aleatória. Algumas palavras próximas, escolhidas a olho, podem ser artefato ou seleção.
- **Dados:** um corpus pequeno produz vetores ruidosos, sobretudo para palavras raras, e só reflete o vocabulário e os usos daquele domínio. Não há evidência para "qualquer domínio", e proximidade estatística não equivale a compreensão.

2. Verificação adicional. Avaliar no espaço original, não na projeção: comparar a similaridade de cosseno entre pares de palavras com julgamentos humanos de similaridade, usando uma lista definida antes de olhar os resultados; repetir o treino com sementes diferentes para checar estabilidade; e testar os embeddings numa tarefa do domínio-alvo e de outro domínio, contra uma linha de base.

3. O que apoiaria e o que não se conclui. Apoiaria a análise: boa concordância com os julgamentos humanos, vizinhos estáveis entre execuções e ganho sobre a linha de base. Mesmo assim, só se conclui que o espaço capta regularidades de coocorrência daquele corpus, para as palavras e tarefas testadas. Não se conclui compreensão, nem bom desempenho em qualquer domínio.

Fundamento. Aulas 3 e 4: trechos sobre visualização de embeddings e interpretação do laboratório. Externo: Wattenberg, Viégas e Johnson (2016), sobre as armadilhas de leitura do t-SNE.

Caso 5: O começo da frase se perdeu

Problema central. Toda a entrada precisa passar por um único vetor de tamanho fixo.

1. O gargalo. No seq2seq clássico, o encoder recorrente lê a frase token a token e entrega ao decoder apenas o último estado oculto. Uma frase de 5 ou de 50 palavras precisa caber na mesma dimensão. Como o estado é reescrito a cada passo, detalhes do início vão sendo diluídos ou sobrescritos, e o decoder não tem outro acesso à entrada. O caminho de gradiente entre a saída e o início da frase também é longo, o que dificulta aprender essas dependências.

2. Como a atenção ajuda. Com atenção, o encoder mantém todos os seus estados, um por posição. A cada passo de geração, o decoder compara seu estado atual com cada estado do encoder, transforma essas pontuações em pesos e monta um vetor de contexto próprio daquele passo. Ao traduzir o começo da frase, pode dar peso alto aos primeiros estados, sem depender de que eles sobrevivam num vetor final.

3. LSTM e processamento sequencial. Não elimina. As portas e a célula de memória da LSTM ajudam a preservar informação e atenuam o desaparecimento do gradiente. Mas o estado no passo t continua dependendo do passo t−1, então o processamento segue sequencial e não paraleliza entre posições. Sem atenção, o gargalo do vetor final também permanece.

Fundamento. Aula 5: trecho sobre seq2seq, LSTM e atenção. Externos: Sutskever, Vinyals e Le (2014); Bahdanau, Cho e Bengio (2015).

Caso 6: Bom no treinamento, ruim na geração

Problema central. Vazamento de informação: o treino deu ao modelo acesso a algo que não existe na geração.

1. Treino, geração e máscara causal. No treino descrito, cada posição podia consultar tokens posteriores, inclusive o próprio alvo. O modelo aprendeu a copiar a resposta, por isso o resultado parecia excelente. Na geração autorregressiva, o futuro ainda não existe: o modelo produz um token por vez vendo só o que já foi gerado. O atalho some e a qualidade cai. A máscara causal impede, dentro da atenção, que a posição i consulte posições j > i, atribuindo peso zero a elas. Assim, o treino usa só o prefixo, como na geração, sem perder o processamento em paralelo.

2. Papéis de Q, K e V. Cada token é projetado por três matrizes aprendidas: a Query representa o que a posição procura; a Key, como cada posição se apresenta para ser encontrada; o Value, o conteúdo que será repassado. A compatibilidade entre Q e K define os pesos, e a saída é a combinação ponderada dos V. Na self-attention, as três vêm da mesma sequência porque ela consulta a si mesma; como as projeções são diferentes, um token pode buscar uma coisa e oferecer outra.

3. Múltiplas cabeças. Cada cabeça tem suas próprias projeções, então a mesma posição pode combinar, em paralelo, informação de padrões diferentes, como vizinhança, concordância ou referência. Essas especializações emergem do treino e podem ser parciais, redundantes ou difíceis de interpretar; nenhuma cabeça tem função fixa garantida.

Fundamento. Aula 6: trecho sobre self-attention, Q/K/V, múltiplas cabeças e máscara causal. Externo: Vaswani et al. (2017).

Caso 7: As peças do Transformer

Problema central. Sem informação de posição, a self-attention trata a frase como um conjunto de palavras.

1. Por que a ordem se perde. Os pesos da atenção dependem só do conteúdo dos tokens (comparação entre Q e K), não de onde estão. As duas frases têm as mesmas palavras e, com atenção bidirecional, cada palavra vê o mesmo conjunto de vizinhos nas duas. Logo, "cachorro" recebe a mesma representação como sujeito e como objeto, e o modelo não tem como saber quem morde quem. Reordenar a entrada apenas reordena as saídas.

2. Funções de cada peça.

- **Informação posicional:** indica onde cada token está (embeddings posicionais absolutos, senoidais ou relativos), tornando a ordem visível ao cálculo.
- **Conexão residual:** soma a entrada de uma subcamada à sua saída, preservando a informação original e facilitando a passagem do gradiente em pilhas profundas.
- **Normalização:** reescala as ativações de cada posição, estabilizando o treino.
- **Feed-forward:** transforma, de forma não linear, cada posição separadamente, depois que a atenção misturou informação entre posições.

3. Por que não se substituem. Cada peça resolve um problema diferente. Residual e normalização estabilizam o treino, mas não criam informação de ordem; o feed-forward atua posição por posição e não troca informação entre tokens; a atenção troca informação, mas é cega à ordem. A informação posicional diz onde cada token está; a máscara diz quem pode consultar quem. Este encoder não usa máscara e ainda assim precisa de posição; e uma máscara causal restringe o acesso, mas não informa explicitamente a posição de cada token.

Fundamento. Aula 7: trecho sobre codificação posicional e componentes do bloco Transformer; Aula 6: trecho sobre máscara. Externo: Vaswani et al. (2017).

Caso 8: Escolher e sustentar o modelo

Problema central. Duas tarefas com formatos de saída diferentes, e um gargalo de memória que cresce com o uso, não com os pesos.

1. Famílias candidatas.

- **Classificação em categorias fixas:** encoder-only (família BERT) com uma camada de classificação. A saída é um rótulo de um conjunto fechado, produzido de uma vez a partir da representação da mensagem inteira, com atenção bidirecional e custo baixo e previsível. Condições: haver exemplos rotulados para ajuste fino, categorias estáveis e mensagens que caibam na janela. Sem rótulos, um decoder com instruções seria alternativa, com mais custo e saída a validar.
- **Assistente com respostas abertas:** decoder-only (família GPT/LLaMA), que gera texto token a token, cada um condicionado ao prefixo. Condições: aceitar o custo da geração e de uma memória proporcional ao contexto.

2. Memória e KV cache. Para não recalcular a atenção sobre todo o histórico a cada novo token, o modelo guarda as chaves e valores (K, V) dos tokens já processados, em cada camada e cabeça. Esse cache cresce com o comprimento do histórico e existe para cada conversa ativa. Os pesos são fixos e compartilhados; o que cresce é o estado por conversa. Mais conversas simultâneas com históricos mais longos esgotam a memória.

3. Três propostas.

- **Compartilhar K/V entre cabeças (MQA/GQA):** reduz o tamanho do cache por token. Limitação: pode perder qualidade e costuma exigir treinar ou adaptar o modelo.
- **Atenção em janelas:** limita quantos tokens anteriores cada posição consulta, o que reduz o número de pares de posições comparados: o custo da atenção deixa de ser quadrático no comprimento do histórico. Limitação: referências antigas, fora da janela, não são consultadas diretamente.
- **Destilação:** treina um modelo menor para imitar o maior, reduzindo parâmetros, memória dos pesos e custo por token. Limitação: o modelo menor pode perder capacidade em tarefas difíceis.

Fundamento. Aula 8: trecho sobre famílias de modelos e eficiência na inferência. Externos: Shazeer (2019); Ainslie et al. (2023); Beltagy, Peters e Cohan (2020); Hinton, Vinyals e Dean (2015).

Variação 1: Caso 1 com falso positivo de alto custo

Caso de origem. Caso 1: classificador de solicitações comuns e urgentes.

Nova condição. Marcar uma solicitação como "urgente" passa a despachar automaticamente uma equipe de campo, e a central tem só duas equipes por turno. Mantêm-se: poucas urgências entre muitas solicitações comuns, o mesmo classificador e uma urgência perdida ainda vai para a fila comum.

O que muda. Na resposta original, o falso positivo custava atenção da equipe; agora consome um recurso escasso e pode deixar uma urgência real sem equipe. A precisão da classe urgente deixa de ser só acompanhamento e vira restrição: os despachos esperados por turno precisam caber na capacidade. A decisão passa de um limiar único para três faixas: alta confiança despacha; confiança intermediária vai para confirmação humana rápida; baixa confiança segue para a fila comum.

O que permanece. A acurácia continua enganosa, porque o desbalanceamento não mudou. O falso negativo continua grave, e a avaliação continua exigindo métricas por classe e matriz de confusão.

Como verificar. Num conjunto de validação rotulado, com a proporção real de classes, levantar precisão e revocação da classe urgente em vários limiares e estimar os despachos por turno de cada um. Apoia a nova conclusão: existir uma faixa em que a revocação atinge o mínimo exigido, os despachos cabem nas duas equipes e o volume intermediário é absorvido pela triagem humana. Põe em dúvida: se, para alcançar revocação aceitável, os despachos falsos excederem a capacidade, o despacho automático não é viável e toda urgência deveria passar por confirmação humana.

Variação 2: Caso 6 com classificação da frase inteira

Caso de origem. Caso 6: modelo que, no treino, consultava tokens posteriores.

Nova condição. O mesmo modelo passa a classificar a mensagem inteira em categorias fixas; em produção, a frase completa está sempre disponível antes da decisão. Mantêm-se: self-attention com múltiplas cabeças, informação de posição e rótulo ausente do texto de entrada.

O que muda. Consultar tokens posteriores deixa de ser vazamento, pois a informação disponível no treino e no uso agora coincide: a frase inteira. A máscara causal deixa de ser necessária e passa a atrapalhar, porque cada posição veria só o prefixo, e palavras do fim não poderiam informar as do começo. A atenção bidirecional, típica de encoder, passa a ser a escolha adequada. Um bom resultado no treino deixa de ser, por si, sinal desse tipo de vazamento.

O que permanece. O princípio de fundo: treinar só com a informação que existirá no uso e nunca expor o alvo. Se o rótulo aparecesse na entrada (por exemplo, uma marca "[URGENTE]" que originou a anotação), o mesmo problema voltaria por outra via. Os papéis de Q, K e V e das múltiplas cabeças não mudam.

Como verificar. Treinar duas versões, com máscara causal e bidirecional, e compará-las num conjunto de validação separado, nas mesmas condições de uso. Apoia: a versão bidirecional igual ou superior, sem grande distância entre treino e validação. Põe em dúvida: desempenho quase perfeito no treino com queda forte na validação (vazamento por outra fonte ou sobreajuste), ou ausência de ganho do bidirecional (o contexto à direita não ajudaria nesta tarefa).

Referências

Materiais da disciplina (Grandes Modelos de Linguagem: do Transformer aos Agentes de IA, V2): Aula 1: tarefas e métricas; Aula 2: tokenização; Aula 3: embeddings e Word2Vec; Aula 4: interpretação do laboratório; Aula 5: recorrência, LSTM e atenção; Aula 6: self-attention e máscara causal; Aula 7: posição e bloco Transformer; Aula 8: famílias de modelos e eficiência.

Fontes externas

- Ainslie, J. et al. (2023). *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. arxiv.org/abs/2305.13245
- Bahdanau, D.; Cho, K.; Bengio, Y. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*. arxiv.org/abs/1409.0473
- Beltagy, I.; Peters, M.; Cohan, A. (2020). *Longformer: The Long-Document Transformer*. arxiv.org/abs/2004.05150
- Devlin, J. et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arxiv.org/abs/1810.04805
- Hinton, G.; Vinyals, O.; Dean, J. (2015). *Distilling the Knowledge in a Neural Network*. arxiv.org/abs/1503.02531
- Jurafsky, D.; Martin, J. H. *Speech and Language Processing* (3ª ed., rascunho). Stanford University. web.stanford.edu/~jurafsky/slp3
- Mikolov, T. et al. (2013). *Efficient Estimation of Word Representations in Vector Space*. arxiv.org/abs/1301.3781
- Petrov, A.; La Malfa, E.; Torr, P.; Bibi, A. (2023). *Language Model Tokenizers Introduce Unfairness Between Languages*. NeurIPS 2023. arxiv.org/abs/2305.15425
- Shazeer, N. (2019). *Fast Transformer Decoding: One Write-Head is All You Need*. arxiv.org/abs/1911.02150
- Sutskever, I.; Vinyals, O.; Le, Q. (2014). *Sequence to Sequence Learning with Neural Networks*. arxiv.org/abs/1409.3215
- Vaswani, A. et al. (2017). *Attention Is All You Need*. arxiv.org/abs/1706.03762
- Wattenberg, M.; Viégas, F.; Johnson, I. (2016). *How to Use t-SNE Effectively*. Distill. distill.pub/2016/misread-tsne

Declaração de uso de IA

Utilizei Claude (claude.ai) para redigir a versão inicial das respostas dos oito casos e das duas variações a partir do enunciado, e Claude Code para apoiar o estudo dos casos, revisar o texto contra o material da disciplina e formatar o documento. Conferi os oito casos, as duas variações e as linhas de fundamentação com base nos materiais das Aulas 1 a 8 (slides das aulas e portal da disciplina) e nas fontes externas listadas em Referências.